0. Set up

Imagine that Princeton came to you for help recruiting its next incoming classes. Due to generous gifts from alumni, the school is expanding its sports programs to allow more students on each team and to add new sports like skiing. The university wants to identify current students who show evidence of sports-related interests, so it can look for high school students with similar interests to recruit.

The university knows that you're taking COS429, and they think you can help. They scraped thousands of images from the Instagram pages of students from the past 5 years, using only images marked "public," and preprocessed this dataset for you to analyze. You have a sneaking suspicion that biases may creep into this analysis.

Your job in this assignment is to train a model that predicts whether an image contains a sports object, then measure its accuracy, precision, and recall overall and across demographic groups. You will analyze the model using **equal opportunity**: among students who truly show sports-related interest, the model should be similarly likely to identify them across groups. In metric terms, this means paying special attention to recall and false-negative-rate gaps.

More precisely, we're going to be using pre-trained features from a ResNet18 model trained on ImageNet to classify images in a dataset called COCO (Common Objects in Context).

Helper file

The starter bundle includes `bias_in_vision_models.py`, which mirrors selected metric and fairness-analysis TODOs from this notebook for staff-side checks. Your notebook analysis should train the sports baseline, compare accuracy, precision, and recall across gender and skin groups, then apply all three fairness interventions to the gender recall gap. The notebook and exported PDF are required for model outputs, tables, plots, and written analysis. Put all written responses in this notebook so they appear in your submitted PDF.

Feature extraction

First, we need to extract features from the pretrained ResNet18 model. We have already computed this for you, but have also provided code for you to look through / recompute to better understand how this works. The necessary data can be found at **annotations_trainval2014.zip**, and **val2014.zip**.

The feature-extraction cells below are optional reference code. You do not need to run them for the assignment if you use the precomputed feature files linked in the next section.

```
import os

print(os.listdir("/content"))
```

```
['.config', 'coco_simplified_features.npy', 'coco_simplified_labels.npz', 'coco_simplified_indices.npy', 'bias_in_vis
```

```
!pip install pycocotools

!cp Downloads/annotations_trainval2014.zip .
!unzip annotations_trainval2014.zip
```

```
Requirement already satisfied: pycocotools in /usr/local/lib/python3.13/dist-packages (2.0.11)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from pycocotools) (2.4.6)
cp: cannot stat 'Downloads/annotations_trainval2014.zip': No such file or directory
unzip:  cannot find or open annotations_trainval2014.zip, annotations_trainval2014.zip.zip or annotations_trainval201
```

```
!cp Downloads/val2014.zip .
!unzip val2014.zip
```

```
cp: Downloads/val2014.zip: No such file or directory
unzip:  cannot find or open val2014.zip, val2014.zip.zip or val2014.zip.ZIP.
```

```
import torch
import torchvision
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, utils
import torchvision.models as models
from PIL import Image
import pickle
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, average_precision_score, confusion_matrix, precision_score, recall_score

from pycocotools.coco import COCO
import json
```

```
annotation_file_path = 'annotations/instances_val2014.json'
coco = COCO(annotation_file_path)
```

```
# For feature extraction, we will ignore categories
# will stitch together categories in complete Dataset object
class COCOSubset(Dataset):

    def __init__(self, image_ids, root = "../../val2014", transform=None):
        self.images = image_ids
        self.root = root
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_id = self.images[idx]

        img_name = "{}/COCO_val2014_{:012d}.jpg".format(self.root, img_id)

        image = Image.open(img_name).convert("RGB")
        #label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return img_id, image


transform = transforms.Compose([transforms.Resize(size=(224, 224)),
                                transforms.ToTensor(),
                                transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                                     std=[0.229, 0.224, 0.225])])


all_img_ids = coco.getImgIds()
dataset = COCOSubset(all_img_ids, root="val2014", transform=transform)
all_loader = DataLoader(dataset, batch_size=256, shuffle=False)
```

```
# Obtain ResNet for feature extraction
class Flatten(torch.nn.Module):
  def __init__(self):
    super().__init__()

  def forward(self, x):
    return x.reshape(x.shape[0], -1)

model = models.resnet18(pretrained=True)
layers = list(model.children())
part_model = torch.nn.Sequential(*layers[:-1])
```

```
# Do feature extraction and save intermediate pkls
all_ids = []
all_features = []

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

part_model.eval()
part_model.to(device)

with torch.no_grad():
  for t, (img_id, x) in enumerate(all_loader):
    x = x.to(device)

    feat = part_model(x).detach().cpu().numpy()
    all_features.append(feat)
```

```
    all_ids.append(img_id.numpy())

    if (t+1)%10==0:
      pickle.dump({"feat": np.concatenate(all_features), "idx":np.concatenate(all_ids)}, open("features{}.pkl".forma
      print(t, "features{}.pkl".format((t+1)//10))
      all_ids = []
      all_features = []
```

```
# Filter data subset by whether the image id has annotations
all_feat = []
all_idx= []

for i in range(1, 16):
  A = pickle.load(open("features{}.pkl".format(i ), 'rb'))
  all_feat.append(A["feat"])
  all_idx.append(A["idx"])


all_feat = np.concatenate(all_feat)
all_idx = np.concatenate(all_idx)


anno = pd.read_csv("images_val2014.csv", index_col=0)

valid_ids = set(anno.index)

mask = np.isin(all_idx, list(valid_ids))

coco_features = all_feat[mask]
coco_indices  = all_idx[mask]

np.save("coco_simplified_features.npy", coco_features)
np.save("coco_simplified_indices.npy", coco_indices)
```

```
# Finally, make target and annotation arrays matched to simplified dataset
coco_ids = np.load("coco_simplified_indices.npy")

target_sport = []
cats_sport = []
genders = []
skins = []

for img_id in coco_ids:
    ann_ids = coco.getAnnIds(imgIds=img_id)
    anns = coco.loadAnns(ann_ids)

    cat_ids = [ann["category_id"] for ann in anns]
    cats = coco.loadCats(cat_ids)

    category_names = [c["name"] for c in cats]
    supercategory_names = [c["supercategory"] for c in cats]

    label_sport = 'sports' in supercategory_names
    cat_sport = category_names[supercategory_names.index('sports')] if label_sport else None

    gender = anno.loc[img_id, 'bb_gender']
    skin = anno.loc[img_id, 'bb_skin']

    target_sport.append(label_sport)
    cats_sport.append(cat_sport)
    genders.append(gender)
    skins.append(skin)


arrays = {
    "target_sport": np.array(target_sport),
    "cats_sport": np.array(cats_sport),
    "genders": np.array(genders),
    "skins": np.array(skins),
}

np.savez_compressed("coco_simplified_labels.npz", **arrays)
```

How to Use Precomputed Features

We have already created files that contain data points with annotations. The notebook will require access to the following 3 files: **coco_simplified_features.npy**, **coco_simplified_indices.npy**, and **coco_simplified_labels.npz**.

[Note: Dropping files into a Colab session repeatedly can be onerous, so one option to sidestep that is to put the files into your Google Drive, and mount your Drive in your Colab session. This option is available from the file explorer in Colab.]

After the required files are available in your Colab session, check the filepaths in the next cell and run it to get data splits and loaders. Do not submit these data files to Gradescope. The loaders will not directly contain annotation values such as skin color, but the train-test splits themselves are matched with each other (so given an image id, you can obtain annotation values).

```
import torch
import torchvision
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, utils
import torchvision.models as models
from PIL import Image
import pickle
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, average_precision_score, confusion_matrix, precision_score, recall_score
```

```
coco_features = np.load("coco_simplified_features.npy")
coco_ids = np.load("coco_simplified_indices.npy")
coco_labels = np.load("coco_simplified_labels.npz", allow_pickle=True)

target_sport = coco_labels["target_sport"]
cats_sport = coco_labels["cats_sport"]
genders = coco_labels["genders"]
skins = coco_labels["skins"]

(
    train_feat,
    val_feat,
    train_ids,
    val_ids,
    train_sport_target,
    val_sport_target,
    train_sport_cat,
    val_sport_cat,
    train_gender,
    val_gender,
    train_skin,
    val_skin,
) = train_test_split(
    coco_features,
    coco_ids,
    target_sport,
    cats_sport,
    genders,
    skins,
    test_size=0.4,
    random_state=42,
)

(
    val_feat,
    test_feat,
    val_ids,
    test_ids,
    val_sport_target,
    test_sport_target,
    val_sport_cat,
    test_sport_cat,
    val_gender,
    test_gender,
    val_skin,
    test_skin,
) = train_test_split(
    val_feat,
```

```
    val_ids,
    val_sport_target,
    val_sport_cat,
    val_gender,
    val_skin,
    test_size=0.5,
    random_state=42,
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
class COCOFeatures(Dataset):
    def __init__(self, img_ids, features, targets, device='cuda'):
        self.img_ids = img_ids
        self.features = features
        self.targets = targets
        self.device = device

    def __len__(self):
        return self.img_ids.shape[0]

    def __getitem__(self, idx):
        img_id = self.img_ids[idx]
        feature = self.features[idx]
        target = self.targets[idx]

        return feature, target, img_id
```

```
batch_size = 64
num_workers = 2


def get_dataloader(ids, feat, target, batch_size=64, num_workers=2, shuffle=False):
    dataset = COCOFeatures(ids, feat, target)
    dataloader = torch.utils.data.DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        num_workers=num_workers,
        pin_memory=True,
    )
    return dataloader


train_sport_loader = get_dataloader(train_ids, train_feat, train_sport_target, shuffle=True)
val_sport_loader = get_dataloader(val_ids, val_feat, val_sport_target)
test_sport_loader = get_dataloader(test_ids, test_feat, test_sport_target)
```

```
from bias_in_vision_models import (
    evaluate_by_group,
    summarize_group_gaps,
    category_recalls,
    lowest_recall_group,
    make_recall_reweighting,
    soft_recall_difference,
    apply_group_thresholds,
)
```

1. Baseline model: sports objects

Sports objects

Model Training.

Train a simple 2-3 layer neural network using the features from a pre-trained ResNet18 model to predict whether an image contains an object with the `sports` label. Everyone will use the sports model so that the fairness analysis and interventions are comparable across submissions.

```
torch.manual_seed(42)
np.random.seed(42)
feature_size = int(np.prod(train_feat.shape[1:]))
num_epochs = 15
learning_rate = 1e-3

def make_sports_model():
    return nn.Sequential(
        nn.Flatten(),
        nn.Linear(feature_size, 64),
        nn.ReLU(),
        nn.Linear(64, 1),
        nn.Sigmoid(),
    ).to(device)


def train_sports_model(model, loader, epochs=15, lr=1e-3,
                       sample_weight_by_id=None, fairness_alpha=0.0,
                       gender_by_id=None):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    losses = []

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        total = 0
        for feat, target, img_id in loader:
            feat = feat.to(device=device, dtype=torch.float32)
            target = target.to(device=device, dtype=torch.float32).reshape(-1, 1)
            scores = model(feat)

            # Normal BCE for the baseline; the weight argument is used for reweighting.
            if sample_weight_by_id is not None:
                weights = torch.tensor(
                    [sample_weight_by_id[int(i)] for i in img_id],
                    dtype=torch.float32, device=device,
                ).reshape(-1, 1)
                loss = nn.BCELoss(weight=weights)(scores, target)
            else:
                loss = nn.BCELoss()(scores, target)

            if fairness_alpha > 0:
                gender = [gender_by_id[int(i)] for i in img_id]
                masks = {
                    'female': np.array([g == 'Female' for g in gender]),
                    'male': np.array([g == 'Male' for g in gender]),
                }
                positive = target.reshape(-1).bool()
                female_positive = positive & torch.as_tensor(masks['female'], device=device)
                male_positive = positive & torch.as_tensor(masks['male'], device=device)
                # Only compare when both groups have positive sports examples in the batch.
                if female_positive.any() and male_positive.any():
                    fair_loss = soft_recall_difference(
                        target, scores, masks, 'female', 'male'
                    )
                    loss = loss + fairness_alpha * fair_loss

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * len(target)
            total += len(target)

        losses.append(total_loss / total)
        if epoch == 0 or (epoch + 1) % 5 == 0 or epoch == epochs - 1:
            print(f'Epoch {epoch + 1}/{epochs}: training loss {losses[-1]:.4f}')
    return losses


def predict_scores(model, features):
    model.eval()
    with torch.no_grad():
        x = torch.as_tensor(features, dtype=torch.float32, device=device)
        return model(x).reshape(-1).cpu().numpy()


model_sports = make_sports_model()
```

```
baseline_losses = train_sports_model(model_sports, train_sport_loader,
                                     epochs=num_epochs, lr=learning_rate)
```

```
Epoch 1/15: training loss 0.2447
Epoch 5/15: training loss 0.1817
Epoch 10/15: training loss 0.1574
Epoch 15/15: training loss 0.1283
```

```
model_sports.eval()
with torch.no_grad():
    x = torch.Tensor(val_feat).to(device)
    val_scores = model_sports(x).squeeze().cpu().numpy()
```

Error analysis.

1. On the validation set, compute the overall accuracy of the sports model as well as the precision and recall scores.
2. Recompute these metrics over four subsets of the dataset: images labelled `female`, images labelled `male`, images labelled as having dark skin, and images labelled as having light skin. Some examples are labelled `Unsure`; include them in the overall metrics, but not in the female/male or dark/light subgroup rows.
3. Report the gender and skin gaps for accuracy, precision, and recall.
4. Compute the recall for different kinds of sports objects.

```
val_groups = {
    'female': val_gender == 'Female',
    'male': val_gender == 'Male',
    'dark_skin': val_skin == 'Dark',
    'light_skin': val_skin == 'Light',
}
comparisons = {'gender': ['female', 'male'],
               'skin': ['dark_skin', 'light_skin']}
metric_names = ['accuracy', 'precision', 'recall']


def measure_scores(scores):
    group_metrics = evaluate_by_group(val_sport_target, scores, val_groups)
    gaps = summarize_group_gaps(group_metrics, comparisons)
    return group_metrics, gaps


def show_metrics(group_metrics, gaps):
    rows = []
    for group in ['overall', 'female', 'male', 'dark_skin', 'light_skin']:
        row = {'Group': group, 'N': group_metrics[group]['n']}
        row.update({m.title(): 100 * group_metrics[group][m] for m in metric_names})
        rows.append(row)
    display(pd.DataFrame(rows).round(2).set_index('Group'))

    gap_rows = []
    for name in ['gender', 'skin']:
        row = {'Comparison': name}
        row.update({m.title() + ' gap (pp)': 100 * gaps[name][m] for m in metric_names})
        gap_rows.append(row)
    display(pd.DataFrame(gap_rows).round(2).set_index('Comparison'))


def compare_metrics(before_metrics, before_gaps, after_metrics, after_gaps, title):
    print(title + ': before/after metrics (%)')
    rows = []
    for group in ['overall', 'female', 'male', 'dark_skin', 'light_skin']:
        for metric in metric_names:
            rows.append({'Group': group, 'Metric': metric,
                         'Before (%)': 100 * before_metrics[group][metric],
                         'After (%)': 100 * after_metrics[group][metric]})
    display(pd.DataFrame(rows).round(2).set_index(['Group', 'Metric']))

    print('Demographic gaps (percentage points)')
    rows = []
    for group in ['gender', 'skin']:
        for metric in metric_names:
            rows.append({'Comparison': group, 'Metric': metric,
                         'Before (pp)': 100 * before_gaps[group][metric],
                         'After (pp)': 100 * after_gaps[group][metric]})
    display(pd.DataFrame(rows).round(2).set_index(['Comparison', 'Metric']))
```

```
male_mask = val_groups['male']
val_male_subset = val_scores[male_mask]
baseline_metrics, baseline_gaps = measure_scores(val_scores)
male_acc = baseline_metrics['male']['accuracy']
male_prec = baseline_metrics['male']['precision']
male_rec = baseline_metrics['male']['recall']
print('Male accuracy, precision, recall:', male_acc, male_prec, male_rec, sep=' ')

show_metrics(baseline_metrics, baseline_gaps)
```

```
Male accuracy, precision, recall:  0.9167270094134685 0.8922345483359746 0.9229508196721311
```

Group	N	Accuracy	Precision	Recall
overall	2985	92.43	87.19	92.14
female	665	92.63	82.61	89.94
male	1381	91.67	89.22	92.30
dark_skin	202	93.56	91.95	93.02
light_skin	1890	92.96	88.14	92.50

Comparison	Accuracy gap (pp)	Precision gap (pp)	Recall gap (pp)
gender	0.96	6.61	2.35
skin	0.60	3.81	0.52

```
# Hint: first find distinct categories within sports objects from train_sport_cat and val_sport_cat
baseline_object_recalls = category_recalls(val_sport_cat, val_scores)
display(pd.DataFrame([
    {'Sports object': name, 'Recall (%)': 100 * recall}
    for name, recall in baseline_object_recalls.items()
]).round(2).set_index('Sports object'))


def compare_sports_objects(before_scores, after_scores):
    before = category_recalls(val_sport_cat, before_scores)
    after = category_recalls(val_sport_cat, after_scores)
    display(pd.DataFrame([
        {'Object': name, 'Before (%)': 100 * before[name],
         'After (%)': 100 * after[name]}
        for name in before
    ]).round(2).set_index('Object'))
```

Sports object	Recall (%)
baseball bat	92.59
baseball glove	94.44
frisbee	87.76
kite	87.93
skateboard	84.62
skis	99.12
snowboard	96.61
sports ball	95.43
surfboard	92.09
tennis racket	94.74

2. Making the sports model more fair

a. Equal-Opportunity Fairness Analysis

Use the principle of **equal opportunity** to analyze the sports model in about 150-200 words. For this assignment, equal opportunity means that among students who truly show evidence of sports-related interest, the model should be similarly likely to identify them across demographic groups.

Use your measured accuracy, precision, and recall. Focus especially on recall or false-negative-rate gaps across gender, since a false negative means a potentially interested student is missed. You should also discuss the skin-group metrics you measured.

Sports Model Equal opportunity means that, among images genuinely containing sports objects, the model should identify them at similar rates across demographic groups. Our baseline model achieved 92.43% overall accuracy, 87.19% precision, and 92.14% recall. For female-labelled images, recall was 89.94%, compared with 92.30% for male-labelled images, a 2.35-percentage-point gender recall gap. This means the model misses a larger share of genuinely sports-positive female-labelled images. In the recruitment scenario, these false negatives could cause interested students to be overlooked.

Accuracy was slightly higher for female-labelled images (92.63%) than male-labelled images (91.67%), while precision was lower (82.61% versus 89.22%). This shows why accuracy alone cannot establish fairness. For skin groups, dark-skin images had 93.56% accuracy, 91.95% precision, and 93.02% recall; light-skin images had 92.96%, 88.14%, and 92.50%, respectively. The skin recall gap was only 0.52 points, although the precision gap was 3.81 points. These are differences in this dataset's predictions, not direct measures of students' interests. Therefore, I would focus on reducing the gender recall gap while monitoring accuracy and precision tradeoffs.

b. Intervening on the sports model

In this section, apply **all three** interventions below to the sports model: reweighting, changing the loss function, and changing the prediction threshold.

Use **gender recall gap** as the shared target metric. This is equivalent to targeting the gender false-negative-rate gap, because lower recall means more false negatives among students who truly show sports-related interest. Continue to report skin-group metrics as secondary diagnostics.

For every intervention, include your code, a before/after metrics table, and a short written interpretation in this notebook so it appears in your submitted PDF. Repeat the same before/after error analysis:

1. Overall accuracy, precision, and recall.
2. Female and male accuracy, precision, and recall.
3. Dark-skin and light-skin accuracy, precision, and recall.
4. Gender and skin gaps for accuracy, precision, and recall.
5. Sports-object recalls when useful for explaining the result.

Success means reducing the gender recall gap. Other metrics may improve, stay similar, or get worse. After each intervention, write a short reflection explaining what worked and what did not. If the target gap decreased, discuss the tradeoffs you observed. If it did not, report that clearly and explain what changed instead.

When training models to be more fair, we can intervene in different stages of the machine learning pipeline:

1. Pre-processing algorithms intervene by adjusting the input data. One method is to reweight different data points.
2. In-processing algorithms intervene during training, for example by adding a fairness term to the loss function.
3. Post-processing algorithms use a trained model and adjust the prediction boundary in different ways.

Reweighting a dataset.

A simple way to reweight different elements during training is to use the `weight` parameter within the `BCELoss()` (https://docs.pytorch.org/docs/stable/generated/torch.nn.BCELoss.html).

For this assignment, use the same train/validation split as the baseline. Identify the gender group with lower baseline recall in the sports model, then upweight positive sports training examples from that group. For example, if the sports model has lower recall on images labelled female, give positive female sports examples a weight $\alpha > 1$ and keep other examples at weight 1.

Repeat the before/after error analysis described above, and report whether the gender recall gap decreased.

```
# Hint: the dataloader returns img_id, which you can use to match samples to demographics
train_groups = {'female': train_gender == 'Female',
                'male': train_gender == 'Male'}
```

```
lower_gender = lowest_recall_group(baseline_metrics, ['female', 'male'])
reweight_alpha = 3.0
weights = make_recall_reweighting(
    train_sport_target, train_groups, lower_gender, weight=reweight_alpha)
weights_by_id = {int(img_id): float(w) for img_id, w in zip(train_ids, weights)}

torch.manual_seed(42)
model_reweighted = make_sports_model()
reweighted_losses = train_sports_model(
    model_reweighted, train_sport_loader, epochs=num_epochs,
    lr=learning_rate, sample_weight_by_id=weights_by_id)
reweighted_scores = predict_scores(model_reweighted, val_feat)
reweighted_metrics, reweighted_gaps = measure_scores(reweighted_scores)
print(f'Upweighted positive {lower_gender} sports examples by factor {reweight_alpha}.')
compare_metrics(baseline_metrics, baseline_gaps,
                reweighted_metrics, reweighted_gaps, 'Pre-processing')
print('Per-object sports recall:')
compare_sports_objects(val_scores, reweighted_scores)
```

```
Epoch 1/15: training loss 0.2570
Epoch 5/15: training loss 0.1902
Epoch 10/15: training loss 0.1647
Epoch 15/15: training loss 0.1375
Upweighted positive female sports examples by factor 3.0.
Pre-processing: before/after metrics (%)
```

Group	Metric	Before (%)	After (%)
overall	accuracy	92.43	91.89
	precision	87.19	85.64
	recall	92.14	92.61
female	accuracy	92.63	91.88
	precision	82.61	78.89
	recall	89.94	92.90
male	accuracy	91.67	91.31
	precision	89.22	88.52
	recall	92.30	92.30
dark_skin	accuracy	93.56	91.58
	precision	91.95	88.76
	recall	93.02	91.86
light_skin	accuracy	92.96	92.43
	precision	88.14	86.39
	recall	92.50	93.25

```
Demographic gaps (percentage points)
```

Comparison	Metric	Before (pp)	After (pp)
gender	accuracy	0.96	0.57
	precision	6.61	9.63
	recall	2.35	0.60
skin	accuracy	0.60	0.85
	precision	3.81	2.38
	recall	0.52	1.39

```
Per-object sports recall:
```

Object	Before (%)	After (%)
baseball bat	92.59	93.83
baseball glove	94.44	97.22
frisbee	87.76	89.80
kite	87.93	93.10
skateboard	84.62	85.71
skis	99.12	99.12
snowboard	96.61	93.22
sports ball	95.43	95.43
surfboard	92.09	89.93
tennis racket	94.74	95.61

Given what you just saw about the effects of reweighting, write about 50 words explaining whether this made the sports model more fair under equal opportunity.

Upweighting positive female sports examples reduced the gender recall gap from 2.35 to 0.60 percentage points, improving equal opportunity. Female recall rose to 92.90%, while male recall stayed unchanged. However, overall accuracy fell to 91.89%, precision to

85.64%, and the skin recall gap widened to 1.39 points. This shows a fairness tradeoff.

Changing the loss function

We can add a fairness term to a standard loss function during training, to incentivize learning of a model that prioritizes accuracy and fairness. To have the loss be differentiable, use the probabilistic outputs of the classifier.

For this assignment, add a penalty for the absolute difference between the female and male soft recalls. If the target labels are $y_i \in \{0, 1\}$, the sensitive attribute is $a_i \in \{\text{female}, \text{male}\}$, and the model outputs scores s_i, the probability of predicting the positive class is $p_i = \sigma(s_i)$. A soft recall-difference penalty is:

$$L_{\text{fair}} = \left| \frac{\sum_{i|(a_i=\text{female and } y_i=1)} p_i}{\sum_{i|(a_i=\text{female and } y_i=1)} 1 + \epsilon} - \frac{\sum_{i|(a_i=\text{male and } y_i=1)} p_i}{\sum_{i|(a_i=\text{male and } y_i=1)} 1 + \epsilon} \right|$$

Thus, the overall loss becomes:

$$L_{\text{overall}} = BCELoss() + \alpha L_{\text{fair}}$$

where α is a hyperparameter that trades off between prediction performance and fairness. Try at least one reasonable value of α, and report the value you used. Repeat the before/after error analysis described above, and report whether the gender recall gap decreased.

```
gender_by_id = {int(img_id): gender for img_id, gender in zip(train_ids, train_gender)}
fairness_alpha = 0.5

torch.manual_seed(42)
model_fair_loss = make_sports_model()
fair_losses = train_sports_model(
    model_fair_loss, train_sport_loader, epochs=num_epochs, lr=learning_rate,
    fairness_alpha=fairness_alpha, gender_by_id=gender_by_id)
fair_scores = predict_scores(model_fair_loss, val_feat)
fair_metrics, fair_gaps = measure_scores(fair_scores)
print(f'Fairness-loss weight: alpha = {fairness_alpha}')
compare_metrics(baseline_metrics, baseline_gaps, fair_metrics, fair_gaps,
                'In-processing')
print('Per-object sports recall:')
compare_sports_objects(val_scores, fair_scores)
```

```
Epoch 1/15: training loss 0.3118
Epoch 5/15: training loss 0.2418
Epoch 10/15: training loss 0.2179
Epoch 15/15: training loss 0.1935
Fairness-loss weight: alpha = 0.5
In-processing: before/after metrics (%)
```

Group	Metric	Before (%)	After (%)
overall	accuracy	92.43	91.76
	precision	87.19	85.46
	recall	92.14	92.42
female	accuracy	92.63	91.28
	precision	82.61	77.89
	recall	89.94	91.72
male	accuracy	91.67	91.24
	precision	89.22	88.50
	recall	92.30	92.13
dark_skin	accuracy	93.56	92.57
	precision	91.95	88.17
	recall	93.02	95.35
light_skin	accuracy	92.96	92.22
	precision	88.14	86.62
	recall	92.50	92.20

```
Demographic gaps (percentage points)
```

Comparison	Metric	Before (pp)	After (pp)
gender	accuracy	0.96	0.04
	precision	6.61	10.61
	recall	2.35	0.42
skin	accuracy	0.60	0.35
	precision	3.81	1.55
	recall	0.52	3.14

```
Per-object sports recall:
```

Object	Before (%)	After (%)
baseball bat	92.59	95.06
baseball glove	94.44	97.22
frisbee	87.76	85.71
kite	87.93	91.38
skateboard	84.62	84.62
skis	99.12	99.12
snowboard	96.61	94.92
sports ball	95.43	96.00
surfboard	92.09	91.37
tennis racket	94.74	95.61

Given what you just saw about the effects of changing the loss function, write about 50 words explaining whether this made the sports model more fair under equal opportunity.

Adding a soft-recall penalty with α = 0.5 reduced the gender recall gap from 2.35 to 0.42 percentage points, improving equal opportunity. Female recall increased to 91.72%, likely because the loss encouraged similar scores on positive examples. However, overall accuracy and